Архитектурные решения

Английский оригинал: ARCHITECTURE_DECISIONS.md.

Каждая запись фиксирует одну техническую развилку: решение, контекст, альтернативы, почему и чего это стоит. Замена решения означает добавление новой записи, а не правку этой.

ADR-1: Читать захваты через dpkt, а не PyShark

- **Решение.** Использовать dpkt 1.9.8 для PCAP и PCAPNG.
- **Контекст.** Инструмент читает захваты офлайн на целевой машине. PyShark — обёртка над `tshark`, отдельным бинарником.
- **Альтернативы.** PyShark; scapy.
- **Почему.** dpkt читает оба формата на чистом Python без внешнего процесса, поэтому набор зависимостей остаётся устанавливаемым одним `pip`. PyShark поставил бы `tshark` на критический путь машины, которой мы не управляем. scapy тяжелее и медленнее для нагрузки до 250k пакетов и тянет больше зависимостей.
- **Следствия.** Мы сами отвечаем за канальный уровень и IPv4/IPv6 (понимаются Ethernet, loopback и BSD cooked; остальное сообщается диагностиками). Поддержка pcapng ровно настолько хороша, насколько хорош dpkt; корпус и фикстуры её упражняют.

ADR-2: Строить корпус как PCAPNG

- **Решение.** Воспроизводимый корпус и фикстуры дефектов — PCAPNG.
- **Контекст.** Задание допускает PCAP и PCAPNG. Фикстуры порождаются `scripts/generate_corpus.py` и `scripts/generate_fixture.py`.
- **Альтернативы.** Классический PCAP.
- **Почему.** PCAPNG несёт метаданные по каждому интерфейсу и поле комментария к пакету и именно его Wireshark пишет по умолчанию, поэтому корпус совпадает с тем, как выглядит настоящий захват. Парсер читает оба, так что выбор одного ничего не стоит.
- **Следствия.** Любая особенность pcapng в dpkt влияет на весь корпус; фикстуры заодно служат регрессионным тестом парсера. В тестах читается и классический PCAP, чтобы оба пути оставались живыми.

ADR-3: Хранить заметки в SQLite, а не в JSON-файле

- **Решение.** Проект держит `annotations.sqlite` для заметок исследователя.
- **Контекст.** Заметки дописываются по ходу сессии и должны переживать перенос каталога проекта; манифест уже указывает на хранилище.
- **Альтернативы.** JSON- или JSONL-файл рядом с манифестом.
- **Почему.** Дописывание в SQLite атомарно и не перезаписывает растущий файл; поиск по смещению или ключу сессии — это запрос, а не скан. Заметки не часть нормализованного контракта, поэтому более богатое хранилище ничего не стоит на проводе.
- **Следствия.** Каталог проекта держит один бинарный файл, который нужно копировать как есть; архив экспорта несёт его без изменений. Читатель без SQLite не прочтёт заметки, но каждый контрактный артефакт остаётся JSON.

ADR-4: Настольный интерфейс на PySide6, а не веб-интерфейс

- **Решение.** Интерфейс — настольное окно на PySide6.
- **Контекст.** Задание просит локальное настольное приложение и задаёт цель Linux x86-64.
- **Альтернативы.** Локальное веб-приложение (браузерный фронтенд, бэкенд на Python); Tkinter.
- **Почему.** Настольное окно соответствует заданию, работает офлайн без сервера и порта, а PySide6 поставляет Qt в колесе, поэтому системный Qt не нужен. Веб-фронтенд добавил бы сервер, порт и зависимость от браузера без выигрыша для одномашинного инструмента. Tkinter не отрисует плотные табличные и hex-виды, которые нужны заданию, без тяжёлой ручной работы.
- **Следствия.** Тесту интерфейса нужен дисплей; набор использует offscreen-режим Qt и пропускается, когда среды исполнения Qt нет. Упаковка Qt-приложения крупнее, чем чистый Python-CLI.

ADR-5: Фрейминг проверкой трёх чтений поля длины, а не предположением одного

- **Решение.** Шаг фрейминга пробует три чтения поля длины и оставляет только то, которое разбирает каждый поток без остатка байт.
- **Контекст.** Смысл поля длины не записан. Оно может считать нагрузку, нагрузку плюс само поле длины или всё сообщение.
- **Альтернативы.** Предположить обычное чтение «только нагрузка» и двигаться дальше.
- **Почему.** Два неверных чтения не падают громко: они дают правдоподобное первое сообщение, а дальше хаос. Проверка всех трёх на реальных байтах всех десяти потоков (см. `docs/REFERENCE_INVESTIGATION.md`) показывает, что одно чтение разбирает всё, а остальные нет, — это доказательство, а не догадка.
- **Следствия.** Фрейминг должен уметь отвергать кандидата, поэтому кандидат судится по «разбирает каждый поток чисто», а не по «что-то произвёл».

ADR-6: Интерпретации — декларативные правила, а не скрипты

- **Решение.** Интерпретация — версионируемое правило JSON или YAML, применяемое движком.
- **Контекст.** Правило описывает кадр и его поля и должно переиспользоваться на новых данных, версионироваться и передаваться.
- **Альтернативы.** Небольшой Python-скрипт на каждую интерпретацию.
- **Почему.** Правило можно проверить по схеме, версионировать, сравнивать, применять из CLI и поставлять в проекте, ничего не исполняя. Скрипт гибок, но его нельзя так же версионировать или проверять, а применение чужого скрипта к своим данным — риск исполнения кода.
- **Следствия.** Схема правил — потолок: поля плоские, а вложенная или переменной длины структура требует изменения схемы. Этот предел записан в `docs/ROADMAP.md`.

ADR-7: Держать движок внутри процесса, не выгружать в Kaitai

- **Решение.** Применять правила встроенным движком; не генерировать спецификацию Kaitai Struct как путь разбора.
- **Контекст.** Kaitai Struct — декларативный формат, выражающий бинарные раскладки и порождающий парсеры.

- **Альтернативы.** Генерировать Kaitai из правила и разбирать им.
- **Почему.** Движок сохраняет связь провенанса: каждый результат поля несёт диапазон потока, из которого он пришёл, поэтому клик в окне возвращает к исходному пакету. Выгрузка в Kaitai либо потеряла бы эту связь, либо потребовала бы собственного обратного отображения, и она добавляет цепочку генерации кода инструменту, который должен работать офлайн из `pip`.
- **Следствия.** Читатели полей мы поддерживаем сами. Взамен формат результата наш, и провенанс выживает от начала до конца.

ADR-8: По умолчанию в памяти, потоковый режим для больших захватов

- **Решение.** Путь по умолчанию держит захват в памяти; потоковый режим освобождает закрытые сессии по ходу.
- **Контекст.** Эталонный профиль — до 100 МиБ и 250k пакетов. Пик памяти масштабируется с захватом, потому что каждый пакет хранится для провенанса.
- **Альтернативы.** Потоковый режим по умолчанию; отобразить файл в память.
- **Почему.** Режим в памяти делает провенанс тривиальным, а код ясным, и на эталонном профиле он даёт пик 350 МиБ против бюджета 4 ГиБ. Потоковый режим доступен там, где памяти меньше; оба дают байт-в-байт одинаковый выход, что и проверяют тесты.
- **Следствия.** Захват в несколько раз больше требует потокового режима или разбора сессия за сессией. Измеренный компромисс — в `docs/BENCHMARK.md`, следующий шаг записан в `docs/ROADMAP.md`.

ADR-9: Поставлять CLI-конвейер вместе с окном

- **Решение.** У каждой стадии есть точка входа командной строки, а команда конвейера проходит от захвата до отчёта за один шаг.
- **Контекст.** Требование R6 просит переиспользовать интерпретацию вне исходной сессии.
- **Альтернативы.** Собрать только GUI.
- **Почему.** CLI делает инструмент скриптуемым и воспроизводимым, что и нужно рецензенту или более позднему прогону, и он работает без экрана. Окно — для исследования; CLI — для повторения. Оба опираются на один движок, поэтому разойтись не могут.
- **Следствия.** Две поверхности нужно держать согласованными. Тесты гоняют CLI и окно по одному корпусу, чтобы поймать расхождение.

ADR-10: Оценивать чтения полей алгоритмически, не звать модель

- **Решение.** Называть поле `hypothesis` оценкой кандидатов-чтений по поддержке и противоречию на корпусе.
- **Контекст.** Поля, помеченные `hypothesis: true`, имеют угаданный смысл. Задание запрещает выдавать догадку за факт.
- **Альтернативы.** Попросить языковую модель назвать поля по примерам.
- **Почему.** Оценка детерминирована и офлайн, поэтому одни и те же байты дают один и тот же балл каждый раз, что и нужно требованию воспроизводимости. Модель сделала бы результат зависящим от внешнего сервиса и промпта и не проверяемым. `docs/REFERENCE_INVESTIGATION.md` показывает, как оценка выбирает `low_cardinality` для `command` и

`target`, тогда как объявленное чтение `constant` проигрывает, — то есть метод способен противоречить догадке.

- **Следствия.** Метод лишь ранжирует чтения, которые даёт исследователь; он их не выдумывает. Этот пробел — первый пункт в `docs/ROADMAP.md`.

ADR-11: Пропуски — маркеры, никогда не заполняются нулями

- **Решение.** Недостающий сегмент — диагностика `gap` со смещением и длиной; поток не дополняется нулями.
- **Контекст.** Захват может быть неполным, и правило не должно читать выдуманные байты как данные.
- **Альтернативы.** Заполнить дыру нулями, чтобы смещения сошлись.
- **Почему.** Заполнение нулями выдало бы выдуманные байты за наблюдение, что запрещено заданием, а поле над дырой прочло бы правдоподобное значение, которого в захвате нет. Сообщение над пропуском классифицируется как `incomplete`.
- **Следствия.** Потребитель потока обязан явно обрабатывать пропуски. Взамен смещение либо отображается на исходный пакет, либо известно как отсутствующее.

ADR-12: Конфликтующие перекрытия остаются неоднозначными

- **Решение.** Когда два пакета перекрываются разными байтами, сохранять первые байты, а противоречие записывать как `ambiguity`, удерживая обе версии.
- **Контекст.** Ретрансмиссии могут нести изменённые байты; настоящий стек повторит, либо захват несогласован.
- **Альтернативы.** Предпочесть поздний пакет; предпочесть ранний; выбрать эвристикой.
- **Почему.** Любой молчаливый выбор выдал бы одну версию за поток и скрыл конфликт. Задание требует, чтобы неоднозначность была видна. Сохранение первых байтов — объявленное правило, а диагностика несёт альтернативу.
- **Следствия.** Сообщение над неоднозначным диапазоном классифицируется как `ambiguous`, а не `matched`. Окно показывает обе версии.

ADR-13: Язык интерфейса через трансляторы Qt, а не самодельную карту

- **Решение.** Держать каждую строку интерфейса в исходниках на английском, оборачивать в `tr()` и переключать язык через `QTranslator` над скомпилированным каталогом Qt.
- **Контекст.** Окну нужен русский и английский интерфейс, выбираемый во время работы и запоминаемый между запусками.
- **Альтернативы.** Словарь `{исходная: перевод}`, искомый вручную при каждом присваивании; сторонняя библиотека i18n.
- **Почему.** `tr()` — идиоматичный Qt, `pyside6-lupdate` собирает строки, так что ни одна не теряется вручную, а `QTranslator` — механизм, которым Qt перестраивает виджеты. Самодельная карта требовала бы вызова поиска при каждом `setText` и расходилась бы с кодом по мере изменения строк. Словарный подход к тому же не может переименовать виджет, когда тот уже построен.
- **Следствия.** Виджеты, строящие свои подписи сами, предоставляют `retranslate_ui()`; окно вызывает его у каждого зарегистрированного вида при смене языка, поэтому переключение живое и не требует перезапуска. Английский — исходный язык и каталога не требует.

Скомпилированный `.qm` закоммичен, чтобы установленный бинарник работал без набора инструментов Qt. Выбор языка — значение `QSettings` в области `madrigal/protocol-lab`.